

Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs

Aaditya K. Singh¹ DJ Strouse²

Abstract

Tokenization, the division of input text into input tokens, is an often overlooked aspect of the large language model (LLM) pipeline and could be the source of useful or harmful inductive biases. Historically, LLMs have relied on byte pair encoding, without care to specific input domains. With the increased use of LLMs for reasoning, various number-specific tokenization schemes have been adopted, with popular models like LLaMa and PaLM opting for single-digit tokenization while GPT-3.5 and GPT-4 have separate tokens for each 1-, 2-, and 3-digit numbers. In this work, we study the effect this choice has on numerical reasoning through the use of arithmetic tasks. We consider left-to-right and right-to-left tokenization for GPT-3.5 and -4, finding that right-to-left tokenization (enforced by comma separating numbers at inference time) leads to largely improved performance. Furthermore, we find that model errors when using standard left-to-right tokenization follow stereotyped error patterns, suggesting that model computations are systematic rather than approximate. We show that the model is able to convert between tokenizations easily, thus allowing chain-of-thought-inspired approaches to recover performance on left-to-right tokenized inputs. We also find the gap between tokenization directions decreases when models are scaled, possibly indicating that larger models are better able to override this tokenization-dependent inductive bias. In summary, our work performs the first study of how number tokenization choices lead to differences in model performance on arithmetic tasks, accompanied by a thorough analysis of error patterns. We hope this work inspires practitioners to more carefully ablate number tokenization-related choices when working towards general models of numerical reasoning.

¹Gatsby Computational Neuroscience Unit, University College London ²Google DeepMind. Correspondence to: Aaditya Singh <aaditya.singh.21@ucl.ac.uk>.

	8302080	8,302,080
	+ 3529456	+ 3,529,456
	<u>11831536</u>	<u>11,831,536</u>
	L2R Tokenization (standard)	R2L Tokenization (enforced by commas)
GPT 3.5 Accuracy:	75.6%	97.8%
GPT 4 Accuracy:	84.4%	98.9%

Figure 1. Illustrating the dependence of frontier model arithmetic performance on tokenization. We show how using commas can enforce right-to-left (R2L) tokenization for the same addition problem. R2L tokenization leads to improved model performance on both GPT-3.5 and GPT-4 (March 2023 models), which we show is due to tokenization alignment between addends and answer through various controls and error analyses.

1. Introduction

Large language models (LLMs) are often lauded as demonstrating the benefits of end-to-end learning over inductive biases. However, an often overlooked part of the pipeline, preventing it from being end-to-end, is tokenization: the segmenting of an input sequence of bytes into discrete tokens. Tokenization consists of two halves: training, in which a vocabulary of tokens and statistics are learned over a given corpus, and segmenting, where a function uses the trained vocabulary and statistics to map sequences of bytes to tokens. Each tokenization scheme may impart different inductive biases on the model due to the way in which bytes of input sequences are grouped – in this work, we study these tokenization-dependent effects on numerical reasoning in state-of-the-art models (GPT-3.5, GPT-4) by considering the tokenization of numbers in arithmetic problems.

Though many techniques have been proposed for tokenization, the prevailing methods in today’s frontier models are variants of Byte Pair Encoding (BPE) (Gage, 1994; Senrich et al., 2016). BPE is a statistical approach to tokenization that is learnt from a dataset of, in the case of LLMs, text. Intuitively, BPE compresses the dataset by iteratively creating tokens for the most commonly occurring subsequences. Specifically, BPE begins with a token vocabulary consisting of each character in the text (e.g. letters, num-

000	001	002	003	004	005	006	007	008	009	010	011	012	013	014	015	016	017	018	019	020	021	022	023	024	025	026	027	028	029	030	031	032	033	034	035	036	037	038	039	040	041	042	043	044	045	046	047	048	049
050	051	052	053	054	055	056	057	058	059	060	061	062	063	064	065	066	067	068	069	070	071	072	073	074	075	076	077	078	079	080	081	082	083	084	085	086	087	088	089	090	091	092	093	094	095	096	097	098	099
100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149
150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199
200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249
250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299
300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349
350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399
400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449
450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499
500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549
550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599
600	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649
650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699
700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749
750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799
800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849
850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899
900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949
950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999

Figure 2. All 3-digit strings, colored red when the string does not have a corresponding single token in `p50k_base`, the BPE tokenizer for GPT-3. Though there’s some patterns (e.g., nearly all multiples of 10 are present), overall there’s no clear structure. The missing tokens are an artifact of the specific process BPE tokenizers use to establish vocabularies.

bers, punctuation).¹ Using this vocabulary, bigram statistics (i.e. frequencies of pairs) are calculated, and the most common bigram is added to the vocabulary and merged in the dataset. This process is repeated up until a prespecified, maximum vocabulary size is reached. After the tokenizer is learned, tokenization of new text proceeds by iteratively merging characters/tokens in the same order as learned on the training dataset.

Naively applying BPE on internet-scale corpora leads to very idiosyncratic number tokenization (Teknum, 2023). In the training phase, which numbers receive dedicated tokens is very adhoc – for example, 710 may get a dedicated token, while 711 will not (Figure 2). In the segmenting phase, these adhoc tokens will lead to different partitionings of numbers of the same length. In Figure 3, we illustrate the different partitionings of all 4 digit numbers when using the `p50k_base` tokenizer that was used to train GPT-3 (Brown et al., 2020). To control for effects on downstream per-

¹More precisely, single bytes are used to handle multilinguality, but this level of description suffices for our needs.

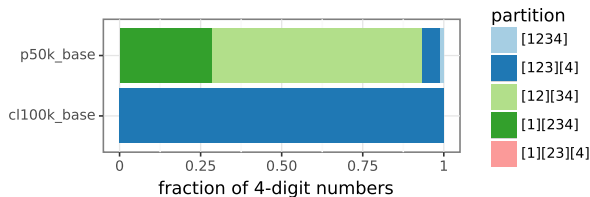


Figure 3. Comparison of how `p50k_base`, the tokenizer for GPT-3, and `cl100k_base`, the tokenizer for GPT-3.5 and GPT-4, segments 4 digit strings into tokens. `cl100k_base` standardized number tokenization to chunks of 3 digits, left-to-right, resulting in all N-digit numbers being segmented the same way.

formance from such idiosyncratic tokenization, prior work (Nye et al., 2021; Zhou et al., 2022) has used formatting, such as spaces or commas, to separate individual digits, ensuring each digit maps to a single token.

Newer models, and their corresponding tokenizers, indicate that LLM practitioners across different labs have also tried to control for idiosyncratic tokenization (Table 1).² The PaLM (Chowdhery et al., 2023), LLaMa (Touvron et al., 2023a) and Mistral³ (Jiang et al., 2023) models switch to single-digit tokenization, similar to that enforced by Nye et al. (2021). Interestingly, GPT-3.5 and GPT-4’s tokenizer, `cl100k_base`, introduces tokens for all 1-, 2-, and 3-digit strings.⁴ Tokenization of numbers by these GPT models defaults to breaking a long number into 3-digit chunks, left-to-right, which we hypothesize (and later show) may create issues for numerical reasoning.

These varied approaches to tokenization by today’s frontier LLMs indicate a lack of convergence in the field on best practices and call for a deeper analysis of (positive or negative) inductive biases imparted by various tokenization schemes. In this work, we provide the first systematic comparison of model performance on the same numerical reasoning tasks with varied tokenization. Specifically, we consider the latest GPT models on few-shot arithmetic tasks.

²A notable exception are Anthropic’s Claude models, which still use pure BPE number tokens (see Appendix A).

³Verified by inspecting the tokens from huggingface https://huggingface.co/docs/transformers/main/en/model_doc/mistral.

⁴This tokenization is enforced by the cryptic `pat_str` parameter in their tokenization library, https://github.com/openai/tiktoken/blob/main/tiktoken_ext/openai_public.py Line 76.

Table 1. Popular LLMs and their number tokenization strategies. BPE = byte pair encoding. L2R = left-to-right.

Model	Strategy
GPT-3 (2020)	pure BPE
GPT-3.5 (2022)	L2R chunks of 3 digits
GPT-4 (2023)	L2R chunks of 3 digits
Claude v2.1 (2023)	pure BPE
Gopher (2021)	pure BPE
Chinchilla (2022)	pure BPE
PaLM (2022)	single digit
GPT-J (2021)	pure BPE
Llama 1 & 2 (2023)	single digit
Mistral (2023)	single digit
OLMo (2024)	pure BPE

We vary the tokenization direction to be the default left-to-right (L2R) or right-to-left (R2L). We find that model accuracy is up to 20% higher when using R2L tokenization (Figure 1, Section 3). We then provide a thorough analysis of error patterns across these two tokenizations (Section 4). We find that the difference in performance between R2L and L2R tokenization in GPT-3.5 can largely be explained by an extremely stereotyped and surprising error pattern (Section 4.3), perhaps indicating the presence of some systematic, but flawed, reasoning. Next, we show that chain-of-thought-inspired approaches, where a model is asked to repeat an input in R2L tokenization, recover the accuracy otherwise lost due to L2R tokenization (Section 5). Finally, we conclude by studying how these effects may change with model version, finding that larger models are better able to override the tokenization-induced effects but, as of yet, unable to eliminate them (Section 6). Overall, we view these results as compelling evidence towards significant tokenization-dependent inductive biases in large language models, and hope they lead model practitioners to conduct careful pre-training ablations with varying tokenization schemes, especially for numerical reasoning.

2. Methods

2.1. Experiment setup

We evaluate GPT models through the Chat Completions endpoint on the OpenAI API⁵ on few-shot addition problems. We control for addend digit length, ranging from 7 to 9 digits (chosen since this way each addend is 3 tokens long). For most experiments, we use 90 random problems, with 10 problems for each addend digit length pair (e.g., 10 problems where the addends are both 7 digits long, 10 problems where the first addend is 7 digits and the second is 8, etc.). For shots, we consider 1-, 2-, 4-, and 8-shots. Shots are sampled randomly for each “query” problem, and are

⁵<https://platform.openai.com/>

provided to the model as a multi-turn dialogue. We control shots to have the same form (digit lengths, tokenization direction, etc.) as the query problem. We use the default system prompt “You are a helpful assistant.” for maximum reproducibility.⁶ Python code for some example 2-shot queries to the model are presented in Appendix E.3 for maximum clarity. We use greedy decoding (temperature=0) in all experiments. Accuracy was computed by extracting numbers from model responses.

Most experiments in the paper are run using the gpt-3.5-turbo-0301 model checkpoint, though in Section 6 we look into how results extend to newer versions of the same model (gpt-3.5-turbo-0613) and to the, presumably larger, GPT-4 models (gpt-4-0314, gpt-4-0613). All code and full results tables can be found at <https://github.com/aadityasingh/TokenizationCounts>.

2.2. Varying L2R vs. R2L tokenization

The ChatCompletion API only allows for input text, not input tokens, so it’s tricky to conduct tokenization-varying experiments. To force the model to use R2L tokenization for numbers, we add commas every 3-digits from the right (see Figure 1). Since the tokenizer doesn’t contain any tokens with numbers and commas, the commas get tokenized separately, effectively enforcing a different segmentation of digits. We use this setting to illustrate our main results, and conduct various controls to ensure that our observed effect is due to tokenization as opposed to other confounds.

3. Right-to-left tokenization improves model performance

3.1. Main results

When using commas to separate digits and enforce R2L tokenization, we observed greatly improved average performance (8-shot result in Figure 1). We found that increasing the number of shots (Figure 4) led to a larger increase for the L2R tokenization (from 68.5% 1-shot to 75.6% 8-shot) than for the R2L tokenization (from 95.6% 1-shot to 97.8% 8-shot) indicating that in-context learning may slightly mitigate the (harmful) bias of L2R tokenization. Given this finding and the plateau-ing in performance with increasing shots, we report only 8-shot results for the remainder of the work as this makes L2R tokenization the most competitive.

3.2. Controlling for comma-based semantic priors

Though this result is already compelling, we realize that commas are often used to separate digits in the manner

⁶We did experiment with other system prompts and found minimal differences (Appendix B).